

A Stratified Domain Model for $\mathbf{U} : \mathbf{U}$ with Π -types: Typing, Validity, Adequacy, and Π -Injectivity

We study a dependent type theory with a universe $\mathbf{U} : \mathbf{U}$ and dependent function (Π) types, in the form mechanised in the `MIN/` Agda development. The main goal is a *syntactic* result — a statement about the conversion (definitional-equality) judgment alone — namely *injectivity of Π* :

$$\Gamma \vdash \Pi(x:A_0) B_0 = \Pi(x:A_1) B_1 : \mathbf{U} \implies \Gamma \vdash A_0 = A_1 : \mathbf{U} \text{ and } \Gamma, x:A_0 \vdash B_0 = B_1 : \mathbf{U}.$$

Because the theory is type-in-type ($\mathbf{U} : \mathbf{U}$) it is logically inconsistent and has no normalisation, so the usual syntactic route to injectivity (via confluence and normal forms) is unavailable. We therefore prove the result *semantically*, by *domain theory*: we build a domain-theoretic model whose elements are finite approximations (finite elements of a Scott domain), define a logical relation linking the model to the syntax, prove *adequacy* (every well-typed term is valid), and then read Π -injectivity off the logical relation by evaluating at the bottom environment. This is the method of [1]; the point of interest is that a purely syntactic fact is obtained from the semantic model.

The presentation here is methodological. The domain is the limit of an increasing chain of finite stages, and we define the information order $\leq$ and the semantic membership $u : a$ accordingly — *by stratification*: a family of relations $\leq_n, \cdot :_n \cdot$ indexed by a stage $n \in \mathbb{N}$, each obtained from the one below, together with a *stability* lemma that lets us pass to the limit. This avoids reasoning through projections (see Section 5.3), and the same idea organises the logical relation.

1 Judgments

We use the two judgment forms (well-formedness of contexts is folded in):

$$\Gamma \text{ ctx}, \quad \Gamma \vdash M : A, \quad \Gamma \vdash M = N : A.$$

Contexts are snoc-lists $\Gamma ::= \cdot \mid \Gamma, x:A$. We write $M\sigma$ for substitution and $B[a]$ for the substitution of a for the last variable. We work in a type-in-type setting, $\mathbf{U} : \mathbf{U}$; the system is therefore logically inconsistent (Girard’s paradox), so we do not address normalisation. Adequacy, Π -injectivity, and subject reduction are statements about the logical relation and are independent of consistency.

2 Typing rules

$$\begin{array}{c}
\text{VAR} \\
\frac{\Gamma \text{ ctx} \quad (x:A) \in \Gamma}{\Gamma \vdash x : A}
\end{array}
\quad
\begin{array}{c}
\text{U} \\
\frac{\Gamma \text{ ctx}}{\Gamma \vdash \mathbf{U} : \mathbf{U}}
\end{array}
\quad
\begin{array}{c}
\text{CONV} \\
\frac{\Gamma \vdash M : A \quad \Gamma \vdash A = B : \mathbf{U} \quad \Gamma \vdash B : \mathbf{U}}{\Gamma \vdash M : B}
\end{array}$$

$$\begin{array}{c}
\text{PI} \\
\frac{\Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U}}{\Gamma \vdash \Pi(x:A) B : \mathbf{U}}
\end{array}
\quad
\begin{array}{c}
\text{LAM} \\
\frac{\Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda(x:A) M : \Pi(x:A) B}
\end{array}$$

$$\begin{array}{c}
\text{APP} \\
\frac{\Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma \vdash f : \Pi(x:A) B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[a]}
\end{array}$$

The context-formation rule is $\cdot \text{ ctx}$ and $\Gamma \vdash A : \mathbf{U} \Rightarrow (\Gamma, x:A) \text{ ctx}$.

3 Conversion rules

$$\begin{array}{c}
\text{REFL} \\
\frac{\Gamma \vdash M : A}{\Gamma \vdash M = M : A}
\end{array}
\quad
\begin{array}{c}
\text{SYM} \\
\frac{\Gamma \vdash M = N : A}{\Gamma \vdash N = M : A}
\end{array}
\quad
\begin{array}{c}
\text{TRANS} \\
\frac{\Gamma \vdash M = N : A \quad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A}
\end{array}$$

$$\begin{array}{c}
\text{CONV-EQ} \\
\frac{\Gamma \vdash M = N : A \quad \Gamma \vdash A = B : \mathbf{U} \quad \Gamma \vdash B : \mathbf{U}}{\Gamma \vdash M = N : B}
\end{array}$$

$$\begin{array}{c}
\text{BETA} \\
\frac{\Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma, x:A \vdash M : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x:A) M) a = M[a] : B[a]}
\end{array}$$

$$\begin{array}{c}
\text{PI-CONG} \\
\frac{\Gamma \vdash A = A' : \mathbf{U} \quad \Gamma, x:A \vdash B = B' : \mathbf{U}}{\Gamma \vdash \Pi(x:A) B = \Pi(x:A') B' : \mathbf{U}}
\end{array}
\quad
\begin{array}{c}
\text{APP-FUN} \\
\frac{\Gamma \vdash f = f' : \Pi(x:A) B \quad \Gamma \vdash a : A \quad \dots}{\Gamma \vdash f a = f' a : B[a]}
\end{array}$$

$$\begin{array}{c}
\text{APP-ARG} \\
\frac{\Gamma \vdash f : \Pi(x:A) B \quad \Gamma \vdash a = a' : A \quad \dots}{\Gamma \vdash f a = f a' : B[a]}
\end{array}$$

$$\begin{array}{c}
\text{FUNEXT} \\
\frac{\Gamma, x:A \vdash f^+ x = g^+ x : B \quad \Gamma \vdash f : \Pi(x:A) B \quad \Gamma \vdash g : \Pi(x:A) B \quad \dots}{\Gamma \vdash f = g : \Pi(x:A) B}
\end{array}$$

Convention. In APP-FUN, APP-ARG, and FUNEXT the ellipsis “...” abbreviates the two formation premises $\Gamma \vdash A : \mathbf{U}$ and $\Gamma, x:A \vdash B : \mathbf{U}$, which every Π -rule carries; we leave them implicit to keep the rules readable.

FUNEXT is the only extensionality principle; it is validated by the model, where two functions with the same finite graph are identified.

4 Head reduction

Single-step head reduction $M \rightsquigarrow_1 N$ is β at the head, closed under the application head:

$$\frac{\text{HR-BETA}}{(\lambda(x:A) M) a \rightsquigarrow_1 M[a]} \qquad \frac{\text{HR-APP} \quad M \rightsquigarrow_1 M'}{M a \rightsquigarrow_1 M' a}$$

We write $M \rightsquigarrow^* N$ for its reflexive–transitive closure. The model tracks only head normal forms: type validity stores $T \rightsquigarrow^* \Pi(x:A) B$, and adequacy reconstructs convertibility from these reductions. Reduction under binders is never used.

5 The domain model

5.1 Finite elements

Codes (finite elements of the Scott domain D) and finite functions are given inductively:

$$u, v, a \in \text{Code} ::= \perp \mid \mathsf{U} \mid \lambda g \mid \Pi(a, f), \quad f, g \in \text{Funcode} ::= [] \mid (u, v) :: f.$$

$\perp$ carries no information, U is the universe code, λg is a function value with finite graph g (a finite list of input/output pairs), and $\Pi(a, f)$ is a Π -type code with domain code a and codomain function f . Each u has a *rank* $\text{rk}(u) \in \mathbb{N}$ (with $\text{rk}(\Pi(a, f)), \text{rk}(\lambda g) = 1 + \max(\dots)$), measuring how many iterative stages are needed to build it.

5.2 Supremum and evaluation

The binary supremum combines elements with the same head constructor and is $\perp$ otherwise:

$$\perp \sqcup u = u, \quad \mathsf{U} \sqcup \mathsf{U} = \mathsf{U}, \quad \lambda g \sqcup \lambda h = \lambda(g ++ h), \quad \Pi(a, f) \sqcup \Pi(b, g) = \Pi(a \sqcup b, f ++ g).$$

We write $u \uparrow v$ (“ u and v are *compatible*”) when they have a common upper bound; on coherent elements this is exactly “same head, and recursively compatible”. Evaluation of a finite function takes the supremum of the outputs whose keys are below the argument:

$$f \cdot u = \bigsqcup \{ v_i \mid (u_i, v_i) \in f, u_i \leq u \}.$$

5.3 Stages: a stratified construction

The domain D is the limit (ideal completion) of an increasing chain $D_0 \subseteq D_1 \subseteq \dots$ of finite approximations. A central methodological point is that *we do not reason through projections* $p_n : D \rightarrow D_n$: the naive transfer “ $u : a \Rightarrow p_n u : p_n a$ ” *fails*. Instead, every finite element of D is presented as the union of an increasing sequence of finite elements of the D_n , and we work entirely with these finite elements. At each stage D_n is a *conditional sup-semilattice* (compatible elements have a least upper bound).

Concretely we define, by induction on the stage n , a relation $\leq_n$ on codes and a relation $\cdot :_n \cdot$ (“ $u :_n a$ ”), each defined “extensionally” from the level below. The level- $(n+1)$ clauses mention only level- n data, so the definitions are structurally recursive in n . For example, type membership at a Π -code is

$$\Pi(a, f) :_{n+1} \mathsf{U} \iff a :_n \mathsf{U} \wedge (\forall x \exists y \leq x. y :_n a \wedge f \cdot x = f \cdot y) \wedge (\forall x. (f \cdot x) :_n \mathsf{U}), \quad (1)$$

expressing that $\Pi(a, f)$ is a well-formed type at stage $n+1$ iff its domain is a type, its graph is determined by finitely many points each lying in a , and all its outputs are types — all checked one stage down. The order on functions is likewise extensional, $f \leq_n g \iff \forall x. f \cdot x \leq_n g \cdot x$.

Lemma 1 (Stability). *For codes u, a with ranks bounded by n ,*

$$u \leq_n v \iff u \leq_{n+1} v \quad \text{and} \quad u :_n a \iff u :_{n+1} a.$$

Stability is proved by induction on n , using that a function value has a finite representation $(u_1, v_1), \dots, (u_m, v_m)$ with $u_i :_n a$ and $v_i :_n \mathbf{U}$. By stability the stage stops mattering above the rank, and we define the limit relations

$$u \leq v := u \leq_n v \quad \text{and} \quad u : a := u :_n a \quad \text{for any } n > \max(\text{rku}, \text{rk}v), \quad n > \max(\text{rku}, \text{rka}),$$

which are well-defined by Lemma 1. Since each relation is given by recursion on the stage n , the construction is manifestly well-founded.

5.4 The information order $\leq$

Unfolding the limit, $\leq$ is the expected structural order: $\perp \leq u$ for all u ; $\mathbf{U} \leq \mathbf{U}$; on compound codes

$$\lambda g \leq \lambda h \iff \forall (u_i, v_i) \in g. v_i \leq h \cdot u_i, \quad \Pi(a, f) \leq \Pi(b, g) \iff a \leq b \wedge f \leq g,$$

and codes with different heads are incomparable.

5.5 Coherence

A finite function is *coherent* when compatible keys force compatible values: $\text{Coh}(g)$ requires that each key and value be coherent and non- $\perp$, and that $u_i \uparrow u_j \Rightarrow v_i \uparrow v_j$. On codes, $\text{Coh}(\Pi(a, f)) = \text{Coh}(a) \wedge \text{Coh}(f)$, while $\perp$ and $\mathbf{U}$ are always coherent. Coherence makes evaluation monotone: $u \leq v$ and $\text{Coh}(g)$ imply $g \cdot u \leq g \cdot v$.

5.6 The membership relation $u : a$

The relation $u : a$ (“the finite element u approximates an element of the type a ”) unfolds, on heads, to:

$$\begin{aligned} \perp : a &\iff a : \mathbf{U} & (\perp \text{ approximates every type}), \\ \mathbf{U} : \mathbf{U} &\iff \top, \\ \Pi(a, f) : \mathbf{U} &\iff a : \mathbf{U} \wedge (\forall (u_i, v_i) \in f. u_i : a \wedge v_i : \mathbf{U}) \wedge \text{Coh}(f), \\ \lambda g : \Pi(a, f) &\iff (\forall (u_i, v_i) \in g. u_i : a \wedge v_i : f \cdot u_i) \wedge \text{Coh}(g) \wedge \Pi(a, f) : \mathbf{U}, \end{aligned}$$

all other head combinations being trivial. The last clause is the meaning of “ λg is a function in $\Pi(a, f)$ ”: every entry of its graph sends a domain point $u_i : a$ to a value v_i of the corresponding fibre $f \cdot u_i$, the graph is coherent, and $\Pi(a, f)$ is itself a type.

5.7 Main properties

All of the following are established in the formalisation:

- **Order.** $\leq$ is reflexive on coherent codes, transitive, and $\perp$ is least. Compatible codes have a least upper bound: if $u \uparrow v$ then $u \leq u \sqcup v$, $v \leq u \sqcup v$, and $u \sqcup v$ is below any common bound. (**Comp-down**) $u \leq u'$ and $u' \uparrow v$ imply $u \uparrow v$. Evaluation is monotone in both function and argument.
- **Type-hood.** $\perp : a \iff a : \mathbb{U}$, i.e. a is a type.
- **Compatible supremum.** If $u_1 : a$ and $u_2 : a$ and $u_1 \uparrow u_2$, then $u_1 \sqcup u_2 : a$. (Membership is closed under conditional sups; this is the semilattice property at the membership level.)
- **Monotonicity.** If $u : a$ and $a \leq b$ and $b : \mathbb{U}$, then $u : b$. (A member of a smaller type is a member of a larger one.)
- **Evaluation in $\mathbb{U}$.** If f 's values lie in $\mathbb{U}$ over a and $x : a$, then $f \cdot x : \mathbb{U}$.

6 The logical relation (validity)

The logical relation connects the model to the syntax. It is built from four mutually-defined predicates, again *stratified by a stage n* (the **ValidityStratified/GoodStage** construction) and collapsed by a stability lemma, so that it too is structurally recursive on n .

6.1 Term and type validity

- **Val $\Gamma M A u a$:** the term $M : A$ in Γ is valid at value code u and type code a .
- **TVal $\Gamma M a$:** the type M is valid at code a .
- **EqVal, EqTVal:** the corresponding equality (two-sided) relations.

The defining clauses mirror the membership relation. At the universe, **Val $\Gamma M A u \mathbb{U}$** (for $u \in \{\mathbb{U}, \Pi(b, f)\}$) asks for **TVal $\Gamma A \mathbb{U}$** and **TVal $\Gamma M u$** . At a Π -code, **TVal $\Gamma M \Pi(b, f)$** unpacks to a head reduction $M \rightsquigarrow^* \Pi(x:A) B$ with $\Gamma \vdash A : \mathbb{U}$, $\Gamma, x:A \vdash B : \mathbb{U}$, domain validity **TVal $\Gamma A b$** , and an *edge function*: for every selection (u, v) from f and well-typed argument N with **Val $\Gamma N A u b$** , one has **TVal $\Gamma B[N] v$** . Dually, **Val** at $(\lambda g, \Pi(b, f))$ asks, for every selection (u, v) from g and such N , that **Val $\Gamma (M N) B[N] v (f \cdot u)$** . A *selection* of f takes or skips each entry, combining the taken pairs by $\sqcup$, and yields (u, v) with $u \leq f \cdot u$ built from subterms of f .

6.2 Stages and stability for validity

As in Section 5.3, **Val**, **TVal**, **EqVal**, **EqTVal** are defined by induction on n . One proves, by induction on n , that if the four relations satisfy all the required properties at stage n then they satisfy them at stage $n+1$; among these properties is *monotonicity of type validity*: if **TVal $\Gamma A a$** and $a' \leq a$ with $a' : \mathbb{U}$, then **TVal $\Gamma A a'$** . Monotonicity then drives stability: for a at stage n , **TVal _{n} $\Gamma A a \iff$ TVal _{$n+1$} $\Gamma A a$** , and similarly for the other three. The collapsed relations are the stage limits. The edge functions hold **Val** only as *stored data* — hypotheses of the form “for all N with **Val $\Gamma N A u b$, ...**” — never inspecting it, so the records are well-formed.

6.3 Main properties

- **Monotonicity (down/up).** Validity transports along $\leq$ on the type code, in both directions where the codes are types.
- **Supremum.** Validity is closed under compatible sups of codes, matching the membership-level property.
- **Reflexivity / symmetry / transitivity** of the equality relations EqVal , EqTVal , and the bundling that exposes the syntactic $\text{HAS_TYPE}/\text{CONV_TM}$ witnesses at the leaves.
- **Head expansion.** Validity is preserved by head reduction of the subject (used in the β case of adequacy).

7 Adequacy

Theorem 1 (Adequacy). *If $\Gamma \vdash M : A$ then, for any coherent environment ρ with a valid substitution σ into a context H , and codes u, a such that M evaluates to u at ρ , A evaluates to a at ρ , and $u : a$, we have $\text{Val } H \ \sigma(M) \ \sigma(A) \ u \ a$.*

The proof is by induction on the typing derivation, with a parallel statement for conversion ($\Gamma \vdash M = N : A$ implies EqVal) and a two-substitution variant (M under two related substitutions yields EqVal). Injectivity and subject reduction are corollaries, obtained by evaluating at the trivial bottom environment $\rho_\perp$ (under which $\Pi B F$ evaluates to $\Pi(\perp, [])$) and reading the head-reduction and conversion data off the logical relation.

8 Π -injectivity

Theorem 2 (Π -injectivity). *If $\Gamma \vdash A_0 = \Pi(x:B_1) F_1 : \mathbb{U}$, then there exist B_0, F_0 with*

1. $A_0 \rightsquigarrow^* \Pi(x:B_0) F_0$,
2. $\Gamma \vdash B_0 = B_1 : \mathbb{U}$,
3. $\Gamma, x:B_0 \vdash F_0 = F_1 : \mathbb{U}$.

Corollary 1 (Π - Π injectivity). *If $\Gamma \vdash \Pi(x:A_0) B_0 = \Pi(x:A_1) B_1 : \mathbb{U}$ then $\Gamma \vdash A_0 = A_1 : \mathbb{U}$ and $\Gamma, x:A_0 \vdash B_0 = B_1 : \mathbb{U}$.*

Proof outline. By presupposition $\Gamma \vdash A_0 : \mathbb{U}$. Conversion soundness at $\rho_\perp$ transfers the evaluation of $\Pi(x:B_1) F_1$ to $\Pi(\perp, [])$ onto A_0 . Bundled equality adequacy yields EqVal at the code $\Pi(\perp, [])$, whose type-validity component records the head reduction $A_0 \rightsquigarrow^* \Pi(x:B_0) F_0$ together with the domain and codomain conversions. Since Π is a head normal form, reduction uniqueness gives the stated equalities. $\square$

Subject reduction (a corollary, in `MIN/SubjectReduction.agda`): if $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$ then $\Gamma \vdash N : A$. The only nontrivial case is β , where Π -injectivity inverts the typing of the redex's λ -abstraction through the conversion.

References

- [1] T. Coquand and S. Huber, “An adequacy theorem for dependent type theory,” *Archive for Mathematical Logic*, vol. 57, pp. 649–676, 2018.